MODULE 05

Advanced Utility Types

TypeScript ships with a powerful standard library of generic helpers that let you derive new types from existing ones — no copy-paste, no duplication. This module covers the ones you'll use in every React project.

~50 min read | Builds on Modules 2-3 | Used in every TS project

Quick Reference — All Utility Types at a Glance

Here's every utility type in this module. We'll deep-dive each one with examples right after.

Utility Type	What it does	Example
`Partial<T>`	Make all properties optional	`Partial<User>`
`Required<T>`	Make all properties required	`Required<Config>`
`Readonly<T>`	Make all properties read-only	`Readonly<User>`
`Pick<T, K>`	Keep only specified properties	`Pick<User, 'id'|'name'>`
`Omit<T, K>`	Remove specified properties	`Omit<User, 'password'>`
`Record<K, V>`	Typed key-value map	`Record<string, number>`
`Exclude<T, U>`	Remove types from a union	`Exclude<Status, 'error'>`
`Extract<T, U>`	Keep only matching types in union	`Extract<Status, 'ok'|'err'>`
`NonNullable<T>`	Remove null and undefined	`NonNullable<string|null>`
`ReturnType<T>`	Get a function's return type	`ReturnType<typeof getUser>`
`Parameters<T>`	Get a function's parameter types	`Parameters<typeof fn>`
`Awaited<T>`	Unwrap a Promise type	`Awaited<Promise<User>>`

1. Partial<T> and Required<T>

Partial<T> — All Properties Become Optional

The most-used utility type. Perfect for update/patch operations where you only send changed fields, or for form state where fields are filled in gradually:

Partial<T> for patch operations

```
interface User {
  id:       number;   // required
  name:     string;   // required
  email:    string;   // required
  age:      number;   // required
}

// Partial<User> = { id?: number; name?: string; email?: string; age?:
number }

// PATCH endpoint — send only changed fields
async function updateUser(id: number, changes: Partial<User>):
Promise<User> {
  return fetch(`/users/${id}`, {
    method: 'PATCH',
    body: JSON.stringify(changes),
  }).then(r => r.json());
}

updateUser(1, { name: 'Alice' });              // OK — just name
updateUser(1, { email: 'a@a.com', age: 31 }); // OK — email + age
updateUser(1, { unknown: 'field' });           // Error: not in User
```

Required<T> — All Properties Become Required

The opposite of Partial. Useful when you have a config with many optional fields but need a validated, fully-filled version:

Required<T> for validated config

```
interface AppConfig {
  apiUrl?:    string;   // all optional — loaded from env
  timeout?:   number;
  retries?:   number;
  debug?:     boolean;
}
```

```
// After validation, every field must exist
type ValidatedConfig = Required<AppConfig>;
// { apiUrl: string; timeout: number; retries: number; debug: boolean }

function startApp(config: ValidatedConfig): void {
  // No optional chaining needed — everything is guaranteed to exist
  console.log(config.apiUrl.toUpperCase()); // Safe — apiUrl: string
}
```

2. Readonly<T> — Immutable Objects

Makes every property of T read-only. TypeScript will error if anything tries to mutate the object. Great for Redux state, config objects, and constants:

Readonly<T> and readonly arrays

```
interface Point { x: number; y: number; }

const origin: Readonly<Point> = { x: 0, y: 0 };
origin.x = 5;  // Error: Cannot assign to 'x' — it is read-only

// Very common with Redux / Zustand state
type AppState = Readonly<{
  user:     User | null;
  products: readonly Product[];  // readonly array too!
  theme:    'light' | 'dark';
}>;

// readonly arrays — can't push/pop/splice
const items: readonly number[] = [1, 2, 3];
items.push(4);  // Error: push does not exist on readonly number[]
items[0];       // OK — reading is fine
```

3. Pick<T, K> and Omit<T, K> — Reshaping Objects

Pick<T, K> — Keep Only What You Need

Creates a new type with only the properties you specify. Great for when a component or function only needs a subset of a larger type:

Pick<T, K>

```
interface User {
  id:        number;
  name:      string;
  email:     string;
  password:  string;   // sensitive!
  createdAt: Date;
}

// Avatar component only needs id and name
type AvatarProps = Pick<User, 'id' | 'name'>;
// { id: number; name: string }

function Avatar({ id, name }: AvatarProps) {
  return `<img id=${id} alt=${name} />`;
}
```

Omit<T, K> — Remove What You Don't Want

The opposite of Pick — creates a type with everything EXCEPT the listed properties. Perfect for stripping sensitive fields like passwords:

Omit<T, K>

```
// Remove sensitive fields before sending to client
type PublicUser = Omit<User, 'password'>;
// { id: number; name: string; email: string; createdAt: Date }

function getPublicProfile(user: User): PublicUser {
  const { password, ...publicUser } = user;  // JS destructuring
  return publicUser;  // TS verifies this matches PublicUser
}

// Omit multiple fields
type UserSummary = Omit<User, 'password' | 'createdAt'>;
// { id: number; name: string; email: string }
```

> **Pick vs Omit — which to use?**
>
> If you want most of the type and just want to remove a few fields, use Omit. If you only want a small subset of a large type, use Pick. The result is the same — choose whichever makes your intent clearer.

4. Exclude<T, U> and Extract<T, U> — Filtering Unions

While Pick/Omit work on object properties, Exclude and Extract work on union types themselves — filtering which members of a union survive:

Exclude and Extract on unions

```
type Status = 'idle' | 'loading' | 'success' | 'error';

// Exclude<T, U> — remove U from the union T
type ActiveStatus = Exclude<Status, 'idle'>;
// 'loading' | 'success' | 'error'

type NonErrorStatus = Exclude<Status, 'error' | 'idle'>;
// 'loading' | 'success'

// Extract<T, U> — keep ONLY the members that match U
type FinishedStatus = Extract<Status, 'success' | 'error'>;
// 'success' | 'error'

// Also useful with mixed unions
type MixedInput = string | number | boolean | null;
type StringsOnly = Extract<MixedInput, string>;  // string
type NoNull      = Exclude<MixedInput, null>;    // string|number|boolean
```

5. NonNullable<T> — Remove null and undefined

A shortcut for Exclude<T, null | undefined>. Use it when you've already checked that a value exists and want to reflect that in the type:

NonNullable<T>

```
type MaybeUser = User | null | undefined;

type DefiniteUser = NonNullable<MaybeUser>;
// User (null and undefined removed)

// Practical use: after a null check
function processUser(user: MaybeUser): void {
  if (!user) return;
  // TypeScript already knows user is User here (via narrowing)
  // But NonNullable helps when you pass it to another function:
  doSomethingRequiringUser(user);  // TS narrows it automatically
}
```

```
function doSomethingRequiringUser(user: NonNullable<MaybeUser>): void {
  console.log(user.name);  // Safe — null/undefined removed
}
```

6. ReturnType<T> and Parameters<T> — Extracting from Functions

These two are incredibly useful when you need to derive a type from an existing function — without having to manually define that type again. This keeps your code DRY (Don't Repeat Yourself):

ReturnType<T>

ReturnType<T>

```
// Imagine this function lives in another file / library
function getUser() {
  return { id: 1, name: 'Alice', role: 'admin' as const };
}

// Get its return type without importing a separate interface
type User = ReturnType<typeof getUser>;
// { id: number; name: string; role: 'admin' }

// Especially useful with API functions
async function fetchProduct(id: number) {
  return { id, name: 'Laptop', price: 999, inStock: true };
}

// Awaited unwraps the Promise, ReturnType gets the function's return
type Product = Awaited<ReturnType<typeof fetchProduct>>;
// { id: number; name: string; price: number; inStock: boolean }
```

Parameters<T>

Parameters<T>

```
function createPost(title: string, body: string, tags: string[]): void {}

// Get the parameter types as a tuple
type CreatePostArgs = Parameters<typeof createPost>;
// [title: string, body: string, tags: string[]]
```

```
// Access individual params by index
type PostTitle = Parameters<typeof createPost>[0];  // string
type PostTags  = Parameters<typeof createPost>[2];  // string[]

// Real use: wrapping a function and forwarding its args
function createPostWithLog(...args: Parameters<typeof createPost>): void {
  console.log('Creating post:', args[0]);
  createPost(...args);
}
```

7. Awaited<T> — Unwrapping Promises

Awaited extracts the resolved value type from a Promise. Use it when you need the inner type of an async operation:

Awaited<T>

```
type A = Awaited<Promise<string>>;           // string
type B = Awaited<Promise<Promise<number>>>;  // number (unwraps deeply)

async function loadUserAndPosts(userId: number) {
  const user  = await fetch(`/users/${userId}`).then(r => r.json());
  const posts = await fetch(`/posts?user=${userId}`).then(r => r.json());
  return { user, posts };
}

// Get the resolved type without manually writing it
type PageData = Awaited<ReturnType<typeof loadUserAndPosts>>;
// { user: any; posts: any }  (would be typed if fetch was typed)
```

8. Combining Utility Types — Real-World Patterns

Utility types are most powerful when combined. Here's how you'll actually use them in a React TypeScript codebase:

Combining utility types — one source of truth

```
interface User {
  id:        number;
  name:      string;
```

```
  email:     string;
  password:  string;
  role:      'admin' | 'user';
  createdAt: Date;
}

// Derived types — single source of truth
type PublicUser   = Omit<User, 'password'>;
type UserSummary  = Pick<User, 'id' | 'name'>;
type UpdateUser   = Partial<Omit<User, 'id' | 'createdAt'>>;
type CreateUser   = Required<Omit<User, 'id' | 'createdAt'>>;
type FrozenUser   = Readonly<PublicUser>;

// Usage in API layer
async function createUser(data: CreateUser): Promise<PublicUser> { ... }
async function updateUser(id: number, changes: UpdateUser):
Promise<PublicUser> { ... }
function renderUserCard(user: UserSummary): string { ... }

// You define User ONCE. Everything else is derived from it.
// Change one field in User and ALL derived types update automatically.
```

> **The single-source-of-truth principle**
>
> The power of utility types is that you define your base interface ONCE, then derive everything from it. If User changes (e.g. you add a 'phone' field), all your derived types (PublicUser, CreateUser, UpdateUser) update automatically. No copy-paste, no drift.

Module Summary

All the utility types covered in this module:

- `Partial<T>` — all props become optional. Great for PATCH/update operations.
- `Required<T>` — all props become required. Great for validated configs.
- `Readonly<T>` — all props become read-only. Great for Redux state, constants.
- `Pick<T, K>` — keep only specified properties from T.
- `Omit<T, K>` — remove specified properties from T.
- `Exclude<T, U>` — remove union members. Works on union types.
- `Extract<T, U>` — keep only matching union members.
- `NonNullable<T>` — remove null and undefined from a type.
- `ReturnType<T>` — extract a function's return type.

- `Parameters<T>` — extract a function's parameter types as a tuple.
- `Awaited<T>` — unwrap a Promise to get the resolved type.
- Combine utility types to derive all your API/component types from one base interface.

```
>  Next: Module 6
```

React + TypeScript Fundamentals

Everything from Modules 1-5 now applied to React. Typing props, children, events, useState, useRef, component return types, and how to structure a typed React component from scratch.